

Builder Journal

Waqar Haider | Week 1 | AI Engineering

Why did you choose AI Engineering?

I chose AI Engineering because it sits exactly at the intersection I care about most: building software that actually thinks, not just software that follows rules. Studying BS Artificial Intelligence gave me the theory, but it was working on my own projects — especially MediScan, my YOLOv8-based bone fracture detection system — that made it real for me. Seeing a model go from a research paper's idea to something that can look at an X-ray and flag a fracture in under two seconds was the moment I knew I didn't just want to study AI, I wanted to engineer it: take it from notebook to product, with a real interface, a real API, and real users.

What type of AI products do you want to build?

I want to build applied AI products that solve a concrete problem end-to-end — not just models, but full products with a frontend, backend, and a business reason to exist. My interests split across two directions: computer-vision-driven tools (health and diagnostics, like MediScan, or automation like my YOLOv8 person-counting system), and consumer-facing AI platforms (like the ApexAI fitness app concept with pose detection and an AI coach, or Nova, the AI chat interface I'm building). Longer term, I want to build agentic products — tools that don't just answer questions but actually plan and execute multi-step tasks on a user's behalf, since that's clearly where the industry is heading.

What was your biggest learning this week?

This week's biggest shift was moving from thinking about LLMs as single-turn chatbots to understanding them as the reasoning core of an autonomous loop. Researching and writing a report on AI agent architecture forced me to actually understand how planning, tool calling, memory, and multi-agent orchestration fit together — not just as buzzwords, but as concrete components (planner, memory store, tool-calling layer, observation loop) that I can now see reflected in tools like Claude Code and MCP-based integrations. In parallel, running five hands-on prompt engineering experiments (role prompting, chain-of-thought, few-shot, structured JSON output, and prompt optimization) showed me very directly how much output quality changes based on how a prompt is constructed — the same request went from unusable to portfolio-ready just by adding role, audience, format, and constraints.

What challenges did you face?

The main challenge was compression: fitting genuinely complex topics (multi-agent orchestration, memory architectures) into a strict page limit without losing the technical substance. It's easy to write a long explanation; it's much harder to decide what to cut. A second challenge was the prompt optimization experiment itself — my first attempts at "improving" a prompt just made it longer, not better, and I had to consciously separate "more words" from "more useful constraints."

How did you solve them?

For the compression problem, I leaned on diagrams to carry structural information that would otherwise take several paragraphs — a single architecture diagram replaced what would have been half a page of prose. For the prompt optimization problem, I switched strategy: instead of adding more description, I added specific, checkable constraints (word count, audience, tone, format) one at a time and compared outputs, which made it obvious which additions were actually improving quality versus just adding noise.

What are your goals for Week 2?

1. Wire up a real tool-calling agent inside Nova (my chat app) so it can do more than answer — starting with one useful action like web search or a simple task tool. 2. Apply the prompt optimization workflow from this week to the actual system prompts I use in my projects, not just as an exercise. 3. Start experimenting with a basic memory layer (short-term context + a simple persistent store) so Nova can remember things across sessions. 4. Keep pushing portfolio and freelance-facing work forward in parallel, so this technical learning turns into something people can actually see and use.